

Greedy Algorithms

Coin Change, Fractional Knapsack, Job Sequencing with Deadlines, Huffman Coding

Dr. Pradeep Kumar S

1. Coin Change – Supermarket Billing

A supermarket cashier wants to return minimum coins for a customer's balance amount.

Input:

Coins = [1, 2, 5, 10]

Amount = 28

Output:

10 10 5 2 1

Minimum coins = 5

<pre>main.py 1 def greedy_coin_change(coins, amount): 2 selected = [] 3 for coin in sorted(coins, reverse=True): 4 while amount >= coin: 5 selected.append(coin) 6 amount -= coin 7 return selected 8 9 coins = [1, 2, 5, 10] 10 amount = 28 11 result = greedy_coin_change(coins, amount) 12 print(*result) 13 print("Minimum coins =", len(result))</pre>	Run Output 10 10 5 2 1 Minimum coins = 5 --- Code Execution Successful ---
--	---

2. Coin Change – Parking Meter

A parking meter accepts coins of fixed denominations. Find the minimum coins needed.

Input:

Coins = [1, 5, 10, 25]

Amount = 63

Output:

25 25 10 1 1 1

Minimum coins = 6

<pre>main.py 1 def greedy_coin_change(coins, amount): 2 selected = [] 3 for coin in sorted(coins, reverse=True): 4 while amount >= coin: 5 selected.append(coin) 6 amount -= coin 7 return selected 8 9 coins = [1, 5, 10, 25] 10 amount = 63 11 result = greedy_coin_change(coins, amount) 12 print(*result) 13 print("Minimum coins =", len(result))</pre>	Output 25 25 10 1 1 1 Minimum coins = 6 --- Code Execution Successful ---
---	---

3. Coin Change – Ticket Counter

A ticket counter must return change using available coin denominations.

Input:

Coins = [1, 3, 7, 10]

Amount = 24

Output:

10 10 3 1

Minimum coins = 4

<pre>main.py 1 def greedy_coin_change(coins, amount): 2 selected = [] 3 for coin in sorted(coins, reverse=True): 4 while amount >= coin: 5 selected.append(coin) 6 amount -= coin 7 return selected 8 9 coins = [1, 3, 7, 10] 10 amount = 24 11 result = greedy_coin_change(coins, amount) 12 print(*result) 13 print("Minimum coins =", len(result))</pre>	Output 10 10 3 1 Minimum coins = 4 --- Code Execution Successful ---
--	--

4. Fractional Knapsack – Food Delivery Truck

A truck can carry limited weight. Select food packets to maximize profit.

Input:

Weights = [10, 20, 30]

Profits = [60, 100, 120]

Capacity = 50

Output:

Maximum profit = 240

<pre>main.py 1 def fractional_knapsack(weights, profits, capacity): 2 items = sorted(3 zip(weights, profits), 4 key=lambda x: x[1] / x[0], 5 reverse=True 6) 7 total_profit = 0.0 8 for weight, profit in items: 9 if capacity == 0: 10 break 11 if weight <= capacity: 12 total_profit += profit 13 capacity -= weight 14 else: 15 total_profit += profit * (capacity / weight) 16 capacity = 0 17 return total_profit 18 19 weights = [10, 20, 30] 20 profits = [60, 100, 120] 21 capacity = 50 22 result = fractional_knapsack(weights, profits, capacity) 23 print("Maximum profit =", f"{result:g}")</pre>	Output Maximum profit = 240 --- Code Execution Successful ---
--	--

5. Fractional Knapsack – Gold Transport

A courier transports gold items with maximum value under weight limit.

Input:

Weights = [5, 10, 15]

Profits = [30, 40, 45]

Capacity = 20

Output:

Maximum profit = 80

<pre>main.py 1 def fractional_knapsack(weights, profits, capacity): 2 items = sorted(3 zip(weights, profits), 4 key=lambda x: x[1] / x[0], 5 reverse=True 6) 7 total_profit = 0.0 8 for weight, profit in items: 9 if capacity == 0: 10 break 11 if weight <= capacity: 12 total_profit += profit 13 capacity -= weight 14 else: 15 total_profit += profit * (capacity / weight) 16 capacity = 0 17 return total_profit 18 19 weights = [5, 10, 15] 20 profits = [30, 40, 45] 21 capacity = 20 22 result = fractional_knapsack(weights, profits, capacity) 23 print("Maximum profit =", f"{result:g}")</pre>	Output Maximum profit = 85 --- Code Execution Successful ---
---	--

6. Fractional Knapsack – Relief Material Loading

Relief items must be loaded to maximize importance value.

Input:

Weights = [4, 8, 2, 6]

Profits = [20, 40, 14, 30]

Capacity = 10

Output:

Maximum profit = 54

<pre>main.py 1 def fractional_knapsack(weights, profits, capacity): 2 items = sorted(3 zip(weights, profits), 4 key=lambda x: x[1] / x[0], 5 reverse=True 6) 7 total_profit = 0.0 8 for weight, profit in items: 9 if capacity == 0: 10 break 11 if weight <= capacity: 12 total_profit += profit 13 capacity -= weight 14 else: 15 total_profit += profit * (capacity / weight) 16 capacity = 0 17 return total_profit 18 19 weights = [4, 8, 2, 6] 20 profits = [20, 40, 14, 30] 21 capacity = 10 22 result = fractional_knapsack(weights, profits, capacity) 23 print("Maximum profit =", f"{result:g}")</pre>	Output Maximum profit = 54 --- Code Execution Successful ---
--	--

7. Job Sequencing – Software Company Tasks

A company wants to schedule tasks before deadlines to earn maximum profit.

Input:

Jobs = A B C D

Deadlines = [2, 1, 2, 1]

Profits = [100, 19, 27, 25]

Output:

Selected jobs = C A

Maximum profit = 127

<pre>main.py 1 def job_sequencing(jobs, deadlines, profits): 2 tasks = sorted(zip(jobs, deadlines, profits), 3 key=lambda x: x[2], reverse=True) 4 max_deadline = max(deadlines) 5 slots = [None] * max_deadline 6 total_profit = 0 7 8 for job, deadline, profit in tasks: 9 for slot in range(min(deadline, max_deadline) - 1, -1, -1): 10 if slots[slot] is None: 11 slots[slot] = job 12 total_profit += profit 13 break 14 15 return [job for job in slots if job], total_profit 16 17 jobs = ['A', 'B', 'C', 'D'] 18 deadlines = [2, 1, 2, 1] 19 profits = [100, 19, 27, 25] 20 selected, profit = job_sequencing(jobs, deadlines, profits) 21 print("Selected jobs =", *selected) 22 print("Maximum profit =", profit)</pre>	Output Selected jobs = C A Maximum profit = 127 --- Code Execution Successful ---
---	---

8. Job Sequencing – Exam Paper Evaluation

Teachers must evaluate papers before deadlines to maximize reward.

Input:

Jobs = J1 J2 J3 J4 J5

Deadlines = [2, 1, 2, 1, 3]

Profits = [20, 15, 10, 5, 1]

Output:

Selected jobs = J2 J1 J5

Maximum profit = 36

<pre>main.py 1 def job_sequencing(jobs, deadlines, profits): 2 tasks = sorted(zip(jobs, deadlines, profits), 3 key=lambda x: x[2], reverse=True) 4 max_deadline = max(deadlines) 5 slots = [None] * max_deadline 6 total_profit = 0 7 8 for job, deadline, profit in tasks: 9 for slot in range(min(deadline, max_deadline) - 1, -1, -1): 10 if slots[slot] is None: 11 slots[slot] = job 12 total_profit += profit 13 break 14 15 return [job for job in slots if job], total_profit 16 17 jobs = ['J1', 'J2', 'J3', 'J4', 'J5'] 18 deadlines = [2, 1, 2, 1, 3] 19 profits = [20, 15, 10, 5, 1] 20 selected, profit = job_sequencing(jobs, deadlines, profits) 21 print("Selected jobs =", *selected) 22 print("Maximum profit =", profit)</pre>	Output Selected jobs = J2 J1 J5 Maximum profit = 36 --- Code Execution Successful ---
---	---

9. Job Sequencing – Printing Press Orders

A printing press schedules orders based on deadlines and profit.

Input:

Jobs = A B C D E

Deadlines = [2, 1, 2, 1, 3]

Profits = [100, 50, 10, 20, 30]

Output:

Selected jobs = B A E

Maximum profit = 180

<pre>main.py 1 def job_sequencing(jobs, deadlines, profits): 2 tasks = sorted(zip(jobs, deadlines, profits), 3 key=lambda x: x[2], reverse=True) 4 max_deadline = max(deadlines) 5 slots = [None] * max_deadline 6 total_profit = 0 7 8 for job, deadline, profit in tasks: 9 for slot in range(min(deadline, max_deadline) - 1, -1, -1): 10 if slots[slot] is None: 11 slots[slot] = job 12 total_profit += profit 13 break 14 15 return [job for job in slots if job], total_profit 16 17 jobs = ['A', 'B', 'C', 'D', 'E'] 18 deadlines = [2, 1, 2, 1, 3] 19 profits = [100, 50, 10, 20, 30] 20 selected, profit = job_sequencing(jobs, deadlines, profits) 21 print("Selected jobs =", *selected) 22 print("Maximum profit =", profit)</pre>	Output Selected jobs = B A E Maximum profit = 180 --- Code Execution Successful ---
---	---

10. Job Sequencing – Freelance Projects

A freelancer can complete one project per day and wants maximum income.

Input:

Jobs = P1 P2 P3 P4

Deadlines = [1, 1, 2, 2]

Profits = [40, 30, 20, 10]

Output:

Selected jobs = P1 P3

Maximum profit = 60

<pre>main.py 1 def job_sequencing(jobs, deadlines, profits): 2 tasks = sorted(zip(jobs, deadlines, profits), 3 key=lambda x: x[2], reverse=True) 4 max_deadline = max(deadlines) 5 slots = [None] * max_deadline 6 total_profit = 0 7 8 for job, deadline, profit in tasks: 9 for slot in range(min(deadline, max_deadline) - 1, -1, -1): 10 if slots[slot] is None: 11 slots[slot] = job 12 total_profit += profit 13 break 14 15 return [job for job in slots if job], total_profit 16 17 jobs = ['P1', 'P2', 'P3', 'P4'] 18 deadlines = [1, 1, 2, 2] 19 profits = [40, 30, 20, 10] 20 selected, profit = job_sequencing(jobs, deadlines, profits) 21 print("Selected jobs =", *selected) 22 print("Maximum profit =", profit)</pre>	Output Selected jobs = P1 P3 Maximum profit = 60 --- Code Execution Successful ---
--	--

11. Huffman Coding – Text Compression

Generate Huffman codes for characters used in file compression.

Input:

Characters = a b c d e f

Frequencies = [5, 9, 12, 13, 16, 45]

Output:

Huffman Codes:

f: 0

c: 100

d: 101

a: 1100

b: 1101

e: 111

```
main.py Run
1 import heapq
2
3 def huffman_codes(characters, frequencies):
4     heap = []
5     order = 0
6     for ch, freq in zip(characters, frequencies):
7         heapq.heappush(heap, (freq, order, ch))
8         order += 1
9
10    while len(heap) > 1:
11        f1, _, left = heapq.heappop(heap)
12        f2, _, right = heapq.heappop(heap)
13        heapq.heappush(heap, (f1 + f2, order, (left, right)))
14        order += 1
15
16    tree = heap[0][2]
17    codes = {}
18
19    def build(node, code=""):
20        if isinstance(node, str):
21            codes[node] = code or "0"
22            return
23        left, right = node
24        build(left, code + "0")
25        build(right, code + "1")
26
27    build(tree)
28    return codes
29
30 characters = ['a', 'b', 'c', 'd', 'e', 'f']
31 frequencies = [5, 9, 12, 13, 16, 45]
32 order_to_print = ['f','c','d','a','b','e']
33 codes = huffman_codes(characters, frequencies)
34 print("Huffman Codes:")
35 for ch in order_to_print:
36     print(f"{ch}: {codes[ch]}")
```

Output

Huffman Codes:

f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111

--- Code Execution Successful ---

12. Huffman Coding – Message Encoding

A messaging app compresses frequently used symbols using Huffman coding.

Input:

Characters = A B C D

Frequencies = [10, 20, 30, 40]

Output:

Huffman Codes:

D: 0

C: 10

A: 110

B: 111

```
main.py Run
1 import heapq
2
3 def huffman_codes(characters, frequencies):
4     heap = []
5     order = 0
6     for ch, freq in zip(characters, frequencies):
7         heapq.heappush(heap, (freq, order, ch))
8         order += 1
9
10    while len(heap) > 1:
11        f1, _, left = heapq.heappop(heap)
12        f2, _, right = heapq.heappop(heap)
13        heapq.heappush(heap, (f1 + f2, order, (left, right)))
14        order += 1
15
16    tree = heap[0][2]
17    codes = {}
18
19    def build(node, code=""):
20        if isinstance(node, str):
21            codes[node] = code or "0"
22            return
23        left, right = node
24        build(left, code + "0")
25        build(right, code + "1")
26
27    build(tree)
28    return codes
29
30 characters = ['A', 'B', 'C', 'D']
31 frequencies = [10, 20, 30, 40]
32 order_to_print = ['D','C','A','B']
33 codes = huffman_codes(characters, frequencies)
34 print("Huffman Codes:")
35 for ch in order_to_print:
36     print(f"{ch}: {codes[ch]}")
```

Output

Huffman Codes:
D: 0
C: 10
A: 110
B: 111

--- Code Execution Successful ---

13. Huffman Coding – Sensor Data Compression

IoT sensor data symbols must be compressed using minimum bits.

Input:

Characters = x y z w

Frequencies = [7, 8, 14, 25]

Output:

Huffman Codes:

w: 0

z: 10

x: 110

y: 111

<pre>main.py 1 import heapq 2 3 def huffman_codes(characters, frequencies): 4 heap = [] 5 order = 0 6 for ch, freq in zip(characters, frequencies): 7 heapq.heappush(heap, (freq, order, ch)) 8 order += 1 9 10 while len(heap) > 1: 11 f1, _, left = heapq.heappop(heap) 12 f2, _, right = heapq.heappop(heap) 13 heapq.heappush(heap, (f1 + f2, order, (left, right))) 14 order += 1 15 16 tree = heap[0][2] 17 codes = {} 18 19 def build(node, code=""): 20 if isinstance(node, str): 21 codes[node] = code or "0" 22 return 23 left, right = node 24 build(left, code + "0") 25 build(right, code + "1") 26 27 build(tree) 28 return codes 29 30 characters = ['x', 'y', 'z', 'w'] 31 frequencies = [7, 8, 14, 25] 32 order_to_print = ['w','z','x','y'] 33 codes = huffman_codes(characters, frequencies) 34 print("Huffman Codes:") 35 for ch in order_to_print: 36 print(f"{ch}: {codes[ch]}")</pre>	Output Huffman Codes: w: 0 z: 10 x: 110 y: 111 --- Code Execution Successful ---
---	--

14. Coin Change – Vending Machine

A vending machine gives minimum coins as change.

Input:

Coins = [1, 2, 5]

Amount = 11

Output:

5 5 1

Minimum coins = 3

<pre>main.py 1 def greedy_coin_change(coins, amount): 2 selected = [] 3 for coin in sorted(coins, reverse=True): 4 while amount >= coin: 5 selected.append(coin) 6 amount -= coin 7 return selected 8 9 coins = [1, 2, 5] 10 amount = 11 11 result = greedy_coin_change(coins, amount) 12 print(*result) 13 print("Minimum coins =", len(result))</pre>	Output 5 5 1 Minimum coins = 3 --- Code Execution Successful ---
--	---

15. Fractional Knapsack – Warehouse Packing

A warehouse manager selects items to maximize value within capacity.

Input:

Weights = [2, 3, 5, 7]

Profits = [10, 5, 15, 7]

Capacity = 8

Output:

Maximum profit = 30

main.py

Run

```
1 def fractional_knapsack(weights, profits, capacity):
2     items = sorted(
3         zip(weights, profits),
4         key=lambda x: x[1] / x[0],
5         reverse=True
6     )
7     total_profit = 0.0
8     for weight, profit in items:
9         if capacity == 0:
10             break
11         if weight <= capacity:
12             total_profit += profit
13             capacity -= weight
14         else:
15             total_profit += profit * (capacity / weight)
16             capacity = 0
17     return total_profit
18
19 weights = [2, 3, 5, 7]
20 profits = [10, 5, 15, 7]
21 capacity = 8
22 result = fractional_knapsack(weights, profits, capacity)
23 print("Maximum profit =", f"{result:.2f}")
```

Output

Maximum profit = 26.67

--- Code Execution Successful ---